

1. การดึงข้อมูล (Read)

ดึงข้อมูลทั้งหมดจากตาราง:

```
sql
SELECT * FROM ชื่อตาราง;
-- ตัวอย่าง: ดึงข้อมูลทั้งหมดจากตาราง employees
SELECT * FROM employees;
ดึงเฉพาะบางคอลัมน์:
```

```
sql
SELECT คอลัมน์1, คอลัมน์2 FROM ชื่อตาราง;
-- ตัวอย่าง: ดึงเฉพาะชื่อและอีเมล
SELECT first_name, email FROM employees;
กรองข้อมูลด้วยเงื่อนไข (WHERE):
```

```
sql
SELECT * FROM ชื่อตาราง WHERE เงื่อนไข;
-- ตัวอย่าง: หาพนักงานที่อยู่แผนก IT (id = 2)
SELECT * FROM employees WHERE department_id = 2;
```

2. การเพิ่มข้อมูล (Create)

เพิ่มข้อมูลใหม่ลงในตาราง:

```
sql
INSERT INTO ชื่อตาราง (คอลัมน์1, คอลัมน์2) VALUES (ค่า1, ค่า2);
-- ตัวอย่าง: เพิ่มพนักงานใหม่
INSERT INTO employees (first_name, last_name, email)
VALUES ('Somsak', 'Rakkaidee', 'somsak@email.com');
```

3. การแก้ไขข้อมูล (Update)

แก้ไขข้อมูลเดิม:

IMPORTANT

ข้อควรระวัง: ห้ามลืมใส่ WHERE เด็ดขาด! ไม่เช่นนั้นข้อมูลของพนักงานทุกคนจะถูกเปลี่ยนเหมือนกันหมด

```
sql
UPDATE ชื่อตาราง SET คอลัมน์ที่ต้องการเปลี่ยน = ค่าใหม่ WHERE เงื่อนไข;
-- ตัวอย่าง: เปลี่ยนเบอร์โทรของพนักงานที่มี id เท่ากับ 5
UPDATE employees
SET phone = '0812345678'
WHERE id = 5;
```

4. การลบข้อมูล (Delete)

ลบข้อมูลออกจากตาราง:

WARNING

ข้อควรระวัง: เช่นเดียวกับ UPDATE ต้องใส่ WHERE เสมอ ไม่งั้นนั้นข้อมูลจะหายทั้งตาราง!

sql

DELETE FROM ชื่อตาราง WHERE เงื่อนไข;

-- ตัวอย่าง: ลบข้อมูลพนักงานที่ลาออกไปแล้ว (id = 10)

DELETE FROM employees WHERE id = 10;

คำสั่งเพิ่มเติมที่มีประโยชน์

เรียงลำดับข้อมูล (ORDER BY):

ASC = น้อยไปมาก (ค่าเริ่มต้น)

DESC = มากไปน้อย

sql

-- ตัวอย่าง: เรียงชื่อพนักงานตามตัวอักษร ก-ฮ

SELECT * FROM employees ORDER BY first_name ASC;

จำกัดจำนวนข้อมูลที่ดึงมา (LIMIT):

sql

-- ตัวอย่าง: ดึงข้อมูลพนักงานมาแค่ 5 คนแรก

SELECT * FROM employees LIMIT 5;

 คลังคำถาม-ตอบ (Q&A) สำหรับการนำเสนอระบบ HRM

เอกสารนี้รวบรวมคำถามที่คาดว่าจะเจอจากอาจารย์ กรรมการ หรือลูกค้า พร้อมแนวทางการตอบที่ดูเป็นมืออาชีพ สำหรับระบบ Human Resource Management (HRM) ของคุณ

 หมวดที่ 1: ภาพรวมและความเป็นมา (System Overview)

Q1: ทำไมถึงเลือกระบบ HRM มาทำ และระบบนี้ช่วยแก้ปัญหา (Pain Point) อะไร?

แนวการตอบ: "จากเดิมที่การจัดการข้อมูลพนักงาน การขอลางาน หรือการประกาศข่าวสารในองค์กรขนาดเล็กถึงกลาง มักใช้กระดาษหรือ Excel ซึ่งทำให้ข้อมูลตกหล่นและตรวจสอบยาก ระบบนี้จึงถูกพัฒนาขึ้นเพื่อรวมศูนย์ข้อมูล (Centralized Data) ช่วยให้การลางานเป็นระบบ พนักงานเข้าถึงข้อมูลตัวเองได้ และผู้บริหารเห็นภาพรวมขององค์กรได้ทันทีผ่าน Dashboard ครับ/ค่ะ"

Q2: กลุ่มเป้าหมายหลัก (Target User) ของระบบนี้คือใคร?

แนวการตอบ: "แบ่งออกเป็น 2 กลุ่มหลักครับ/ค่ะ:

Admin/HR: สำหรับจัดการข้อมูลพนักงาน, อนุมัติการลา, ออกประกาศ และดูสถิติต่างๆ

Employee (พนักงาน): สำหรับดูข้อมูลส่วนตัว, ยื่นเรื่องลางาน, และรับทราบประกาศจากบริษัท"

 หมวดที่ 2: เทคโนโลยีที่ใช้ (Technology Stack)

Q3: ระบบนี้ใช้เทคโนโลยีอะไรในการพัฒนาบ้าง และทำไมถึงเลือกใช้?

แนวการตอบ: "เราเลือกใช้ Modern Stack เพื่อประสิทธิภาพและความปลอดภัยครับ/ค่ะ:

Frontend & Backend: ใช้ Next.js (React Framework) ร่วมกับ TypeScript ซึ่งช่วยให้เว็บโหลดเร็ว (SEO Friendly) และเขียนโค้ดได้ปลอดภัยขึ้นด้วยการเช็ค Type
Styling: ใช้ Tailwind CSS เพื่อความรวดเร็วในการจัดหน้าตาที่สวยงามและ Responsive
Database: ใช้ MySQL เพราะมีความเสถียร จัดการข้อมูลแบบสัมพันธ์ (Relational) ได้ดี
State Management: ใช้ React Query (TanStack) ช่วยจัดการเรื่องการแคชข้อมูลและลดการโหลดซ้ำซ้อน"

Q4: ทำไมถึงเลือก Next.js แทนที่จะเป็น React ธรรมดา?

แนวการตอบ: "Next.js มีฟีเจอร์ API Routes ในตัว ทำให้เราสามารถเขียน Backend (Server-side) เชื่อมต่อกับฐานข้อมูลได้เลยในโปรเจกต์เดียวกัน โดยไม่ต้องแยกโปรเจกต์ Backend ต่างหาก ช่วยให้การดูแลรักษาง่ายขึ้นครับ/ค่ะ"

🔧 หมวดที่ 3: ฟังก์ชันการทำงาน (Key Features)

Q5: ฟังก์ชันการลางาน (Leave Management) มีขั้นตอนการทำงาน (Flow) อย่างไร?

แนวการตอบ: "พนักงานกรอกฟอร์มขอลางาน -> ระบบจะบันทึกสถานะเป็น 'รอดำเนินการ' (Pending) -> แจ้งเตือนไปยัง Admin -> เมื่อ Admin กดอนุมัติหรือปฏิเสธ สถานะจะอัปเดต และพนักงานสามารถเข้ามาเช็คประวัติการลาของตนเองได้ครับ/ค่ะ"

Q6: ระบบมีฟังก์ชันพิเศษอะไรที่น่าสนใจอีกบ้าง?

แนวการตอบ: "เรามีฟังก์ชันที่ช่วยอำนวยความสะดวก เช่น:

การสร้าง QR Code พนักงาน สำหรับใช้ระบุตัวตน

การ Export ข้อมูลเป็น Excel (.xlsx) เพื่อนำไปประมวลผลต่อ

ระบบพิมพ์บัตรพนักงาน (Print ID Card) ผ่านหน้าเว็บได้ทันที

Dashboard สรุปข้อมูล ที่แสดงผลเป็นกราฟ (Recharts) ดูเข้าใจง่าย"

🔒 หมวดที่ 4: ความปลอดภัยและการจัดการข้อมูล (Security & Data)

Q7: ระบบมีการจัดการสิทธิ์การเข้าใช้งาน (Authorization) อย่างไร?

แนวการตอบ: "เราใช้ JWT (JSON Web Token) ผ่านไลบรารี jose ในการยืนยันตัวตน และเก็บ Token ไว้ใน Cookie ที่ปลอดภัย เมื่อมีการเข้าถึงหน้าต่างๆ ระบบจะเช็ค Role (Admin/Employee) ก่อนเสมอเพื่อป้องกันไม่ให้พนักงานทั่วไปเข้าถึงหน้าจัดการของ Admin ได้ครับ/ค่ะ"

Q8: รหัสผ่านของพนักงานถูกเก็บไว้อย่างไร ปลอดภัยไหม?

แนวการตอบ: "ปลอดภัยแน่นอนครับ/ค่ะ เราใช้ไลบรารี Bcrypt ในการเข้ารหัสผ่าน (Hashing) ก่อนบันทึกลงฐานข้อมูล ทำให้แม้กระทั่งผู้ดูแลระบบก็ไม่สามารถเห็นรหัสผ่านจริงของพนักงานได้"

🚀 หมวดที่ 5: ปัญหาและการต่อยอด (Future Enhancements)

Q9: พบปัญหาอะไรบ้างระหว่างการพัฒนา และแก้ไขอย่างไร?

แนวการตอบ: "เช่น ปัญหาเรื่องการจัดการรูปแบบวันที่ (Date Format) ที่ไม่ตรงกันระหว่าง Input และ Database เราจึงแก้ปัญหาโดยการทำฟังก์ชันจัดรูปแบบวันที่ให้เป็นมาตรฐานเดียวกันทั้งหมด หรือปัญหาเรื่อง TypeScript Error ที่เราไล่แก้เพื่อความปลอดภัยของโค้ดครับ"

Q10: หากมีเวลาเพิ่ม อยากพัฒนาฟังก์ชันอะไรต่อ?

แนวการตอบ:

เพิ่มระบบ Payroll คำนวณเงินเดือนและภาษี

เชื่อมต่อกับระบบ Time Attendance (สแกนนิ้ว/เช็คอินผ่าน GPS)

ระบบแจ้งเตือนผ่าน Line Notify เมื่อมีการขอลางาน

💡 ทริคเล็กๆ สำหรับการนำเสนอ

DEMO ให้เห็นภาพ: อย่าพูดแค่สไลด์ ให้เปิดระบบจริงและลองทำ Flow ให้อู (เช่น ลองกดลางาน แล้วสลับไอดีไปกดอนุมัติ)

มั่นใจในเทคโนโลยี: การใช้ Next.js + TypeScript ถือเป็นเรื่องที่กรรมการมักจะชื่นชม เพราะเป็น

เทคโนโลยีปัจจุบันที่ใช้ในระดับอุตสาหกรรมจริง

คลังคำถาม-ตอบ เชิงเทคนิคระดับลึก (Technical Deep Dive Q&A)

รวบรวมคำถามและคำตอบเจาะลึกด้านสถาปัตยกรรมระบบ โค้ด และฐานข้อมูล สำหรับกรรมการสอบหรือทีมเทคนิค

หมวดที่ 1: สถาปัตยกรรมระบบและการเขียนโค้ด (Code Architecture)

Q1: การเชื่อมต่อฐานข้อมูล (Database Connection) จัดการอย่างไร และทำอย่างไรให้ไม่เกิดปัญหา Connection ค้าง?

แนวการตอบ: "ระบบใช้ไลบรารี mysql2/promise ในการทำ Connection Pool โดยจำกัดการเชื่อมต่อสูงสุดที่ 10 Connections (connectionLimit: 10) ในไฟล์ hrm_db.ts

ครับ/ค่ะ นอกจากนี้เรายังใช้ Global Object เพื่อจำกัดไม่ให้ Next.js สร้าง Pool ขึ้นมาใหม่หลายอันในขั้นตอน Development เวลาแก้โค้ดแล้วระบบรีโหลดตัวเอง ทำให้ทีมงานทรัพยากรได้อย่างคุ้มค่าและลดโอกาสเกิดปัญหา Database Lock"

Q2: ใน Next.js คุณแยกการเขียนโค้ดระหว่างฝั่ง Frontend กับ Backend อย่างไร?

แนวการตอบ: "ใช้ประโยชน์จาก Next.js App Router ครับ/ค่ะ:

ส่วน UI จะอยู่ในโฟลเดอร์ src/app/ (เช่น

leaves/page.tsx

)

ส่วน API (Backend) จะเขียนอยู่ใน src/app/api/ (เช่น

api/leaves/route.ts

) ซึ่งทำงานในฝั่ง Server-Side ทั้งหมด ปลอดภัยจากการดักจับข้อมูล (Interception) และสามารถดึงข้อมูลจาก DB ได้โดยตรง"

Q3: ระบบมีการเก็บประวัติการทำงาน (Audit Logs) อย่างไร?

แนวการตอบ: "มีระบบ logAudit ที่เขียนดักไว้ในทุกๆ Action สำคัญของ API เช่น การเพิ่ม/แก้ไขพนักงาน หรือการอนุมัติการลา โดยจะทำการบันทึก User ID, วันเวลา, และรายละเอียดกิจกรรมลงในฐานข้อมูล [tbl_audit_logs] ทุกครั้งครับ/ค่ะ"

หมวดที่ 2: ระบบฐานข้อมูล (Database & Data Flow)

Q4: ออกแบบตารางในฐานข้อมูล (Database Schema) อย่างไร และมีจุดเชื่อมโยง (Relations) อย่างไรบ้าง?

แนวการตอบ: "ออกแบบฐานข้อมูลแบบ Relational Database โดยตารางหลักๆ เช่น:

tbl_employees (ข้อมูลพนักงาน) เก็บ dept_id เป็น Foreign Key เชื่อมกับ tbl_departments

tbl_leaves (การลางาน) เก็บ emp_id อ้างอิงพนักงานที่ยื่นคำขอ เวลาดึงข้อมูล จะใช้คำสั่ง SQL LEFT JOIN เพื่อรวมข้อมูลพนักงานกับชื่อแผนกขึ้นมาแสดงผลในขั้นตอนเดียว เพื่อลดภาระการ Query หลายครั้งครับ/ค่ะ"

Q5: การจัดการเรื่องการอัปโหลดไฟล์ (เช่น ไฟล์แนบใบรับรองแพทย์เวลาลาป่วย) เก็บข้อมูลอย่างไร?

แนวการตอบ: "เมื่อ Client ยื่นฟอร์มด้วย multipart/form-data ผ่าน API:

โค้ดจะดึงไฟล์ออกจาก req.formData()

ตั้งชื่อไฟล์ใหม่โดยระบุรหัสพนักงานและ Timestamp เพื่อกันชื่อซ้ำ

เขียนไฟล์ลงในระบบ (Disk Storage) ที่โฟลเดอร์ public/uploads/

จากนั้นจะเก็บเฉพาะ File Path (ชื่อไฟล์) ลงในคอลัมน์ attachment ของตาราง tbl_leaves ครับ/ค่ะ"

⚙️ หมวดที่ 3: ตรรกะฟังก์ชันการทำงานเชิงลึก (Logical Functions)

Q6: ขั้นตอน (Workflow) การอนุมัติการลาในโค้ด ทำงานอย่างไร?

แนวการตอบ: "การลามีหลายขั้นตอน (Stage) ในโค้ดจะมีฟิลด์ `current_stage` กำหนดสถานะ เช่น 'Head of Dept', 'Admin', 'Director' โดยเริ่มแรกสถานะจะเป็น 'Pending' ทั้งหมด เมื่อผ่านการอนุมัติจากแต่ละระดับ ข้อมูลใน DB จะอัปเดตสถานะ (เช่น `dept_head_status = 'Approved'`) และเลื่อน `current_stage` ไปสู่ลำดับถัดไป จนกว่าจะผ่านทุกขั้นตอนครับ/ค่ะ"

Q7: การทำระบบแจ้งเตือน (Notifications) ในโปรเจกต์นี้ทำงานอย่างไร?

แนวการตอบ: "ใช้ระบบ Database-driven ครับ/ค่ะ เมื่อเกิดการกระทำใหม่ ๆ (เช่น พนักงานยื่นใบลา) โค้ดใน API จะทำการ INSERT ข้อมูลการแจ้งเตือนลงใน `tbl_notifications` เช่น แจ้งเตือนไปยัง Admin แล้วฝั่ง Frontend จะคอยดึง (Poll) ข้อมูลจากตารางนี้มาแสดงผลเป็นจุดสีแดงแจ้งเตือนให้ผู้ใช้เห็น"

🔒 หมวดที่ 4: ความปลอดภัยและ Performance

Q8: มีวิธีป้องกัน SQL Injection ในโปรเจกต์นี้อย่างไร?

แนวการตอบ: "หลีกเลี่ยงการนำ String ตัวแปรไปเชื่อม (Concatenate) ในคำสั่ง SQL ตรงๆ ครับ/ค่ะ แต่จะใช้ฟีเจอร์ Parameterized Queries (Prepared Statements) ของ `mysql2` โดยใช้เครื่องหมาย `?` แทน แล้วส่งค่าตัวแปรแยกเป็น Array ไปแทนที่ ทำให้ฐานข้อมูลรู้ว่าค่าที่ส่งมาคือข้อมูลดิบ ไม่ใช่ชุดคำสั่ง SQL ครับ/ค่ะ"

1. การเตรียมตัวล่วงหน้า (ห้ามพลาด!)

เปิดระบบทิ้งไว้เลย: ก่อนเริ่มพรินต์ ให้เปิดหน้าเว็บระบบทิ้งไว้ (ห้ามมารัน `npm run dev` ต่อหน้ากรรมกร) และเช็คข้อมูลตัวอย่าง (Mock Data) ให้พร้อม ไม่ควรเดโมด้วยตารางเปล่าๆ

เทคนิคสลับไอดี (Incognito Mode): ระบบของคุณมีการสลับระหว่าง Admin และ พนักงาน

ทริค: ให้เปิดเบราว์เซอร์ปกติเป็น Admin และเปิดเบราว์เซอร์แบบไม่ระบุตัวตน (Incognito) เป็น พนักงานทิ้งไว้ เพื่อที่คุณจะได้เดโมสลับหน้าจอขอลางานและอนุมัติได้อย่างรวดเร็ว โดยไม่ต้องกด Log out ให้เสียเวลา

2. ลำดับการพูด (Presentation Flow) ที่ดี

นาทีกี่ 1-2 (เปิดหัว): อย่าเพิ่งพูดฟังก์ชัน! ให้เริ่มจาก ปัญหา (Pain Point) เช่น "ก่อนหน้านี้บริษัทเราขอลางานผ่านกระดาษ ทำให้ข้อมูลหาย..."

นาทีกี่ 3-7 (เดโมระบบ): เดินเรื่องตาม User Journey (การใช้งานจริง) เช่น:

ในมุมมองพนักงาน: ล็อกอิน ยื่นคำขอลางาน แนบไฟล์ใบรับรองแพทย์

ในมุมมอง Admin: มีแจ้งเตือนเข้ามา ดูประวัติการลา แล้วกด "อนุมัติ"

สรุปผล: โชว์กราฟ Dashboard และลอง Export รายงานเป็น Excel

นาทีกี่สุดท้าย (ปิดท้าย): สรุปว่าระบบนี้ช่วยประหยัดเวลาได้กี่ % และพูดถึงอนาคต (เช่น จะทำระบบ Payroll เพิ่ม)

3. คาถาเอาตัวรอด เมื่อเจอ "คำถามมรณะ"

ถ้าโดนถามถึงฟังก์ชันที่ระบบยังทำไม่ได้ หรือฟังก์ชันที่อาจารย์ถามขึ้นมาสดๆ ห้ามพูดว่า "ทำไม่เป็น" เด็ดขาด ให้ใช้ประโยคทองคำนี้:

💡 "เป็นฟังก์ชันที่ดีและสำคัญมากครับ/ค่ะ ในเวอร์ชันปัจจุบันเราเน้นพัฒนา Core Function หลักให้เสถียรก่อน แต่ฟังก์ชันที่อาจารย์เสนอมายู่ในแผนงานพัฒนาต่อใน Phase ที่ 2 แน่นอนครับ/ค่ะ"